TECHNICAL DESIGN DOCUMENT (TDD)

B2B Distributor Order Management System & Hindi / Hinglish Order Engine

Client: Company A, Jabalpur (4 Packaged Food & Home Care Brands, 1,800 Kirana Shops, 18 Salesmen)
Engineering Stack: Next.js 16 (App Router), TypeScript, Neon PostgreSQL, Drizzle ORM, Vitest
Operational Status: Production Ready • Version 3.2 • 258 Tests (100% Passing) • 100% Scorer Precision (96.9% 95% Wilson LCB)
Repository: https://github.com/manva-niso/Distributor-Order-Management-System.git
Live Application: https://distributor-order-management-system.vercel.app/login

1. What I Built and Why

Company A is an FMCG distributor in Jabalpur handling ~600 daily morning wholesale orders from 1,800 kirana shops. Orders historically arrived via unstructured WhatsApp text messages, photos of paper slips ("parchi"), and Hindi voice notes.

Three Legacy Frictions

- **Manual Re-typing:** Six office clerks spent mornings manually re-typing unstructured WhatsApp orders into Tally ERP.
- **Salesman Stock Blindness:** Field salesmen booked orders without knowing real-time warehouse inventory, causing post-booking stockout cuts.
- **Credit Breaches & Disputes:** Retailers ordered beyond credit limits, and loose pack pricing confusion triggered frequent invoice disputes.

What I Built

A mobile-friendly web application for budget Android smartphones on 3G networks. Retailers can order directly within credit limits or submit unstructured text, slip photos, or voice notes. The system reads dialect text, verifies stock, applies trade offers and GST, and exports 1-tap Tally XML files.

ASSUMPTIONS ** : * 1. `[A31]` Standard Excel Catalogue Format: **I operate under the assumption that the product catalogue and price updates are provided in the standard Excel (.xlsx or .xls) format used in this app. * 2.** `[A32]` Flexible Column Handling:** The Excel reader automatically handles variations in header names (e.g. "Item Name", "Product", "Description") and fills safe defaults for missing values.

2. Architecture & Technology Decisions

Every technology choice balances low-cost smartphone performance, high morning ordering concurrency, and offline-tolerant operations.

Part	What I Used	Why I Chose It & Alternatives Rejected
App & Server	**Next.js 16 (App Router) + TypeScript**	Unified repository for user screens and backend API routes with shared types; rejected separate Vite/Express due to deployment sync issues.
Look & Feel	**Tailwind CSS v4**	Lightweight stylesheet (< 18 KB compressed) enforcing large touch targets (≥ 48 × 48 px) and high-contrast card borders readable in sunlight.
Database	**PostgreSQL on Neon + Drizzle ORM**	ACID-compliant relational cloud database handling morning order bursts and preserving saved orders across serverless restarts.
Sign-In & Security	**Mobile OTP + Signed Session Cookies**	Passwordless login for busy shopkeepers; keeps users signed in for 7 days and identifies roles (retailer, salesman, desk clerk).
Order Reading Engine	**Rules-First + Scoring Model + Gemini AI Fallback**	5-tier hybrid cascade: instant rules run in < 5 ms, 12-feature scoring model infers in 0.00048 ms, and Gemini AI resolves hard dialect cases.
Automated Testing	**Vitest Test Runner**	Fast test runner executing all 258 tests across 31 test files in under 5 seconds with zero internet required.

3. How Data is Stored

The database partitions data into two logical groups: user identity and wholesale commercial records.

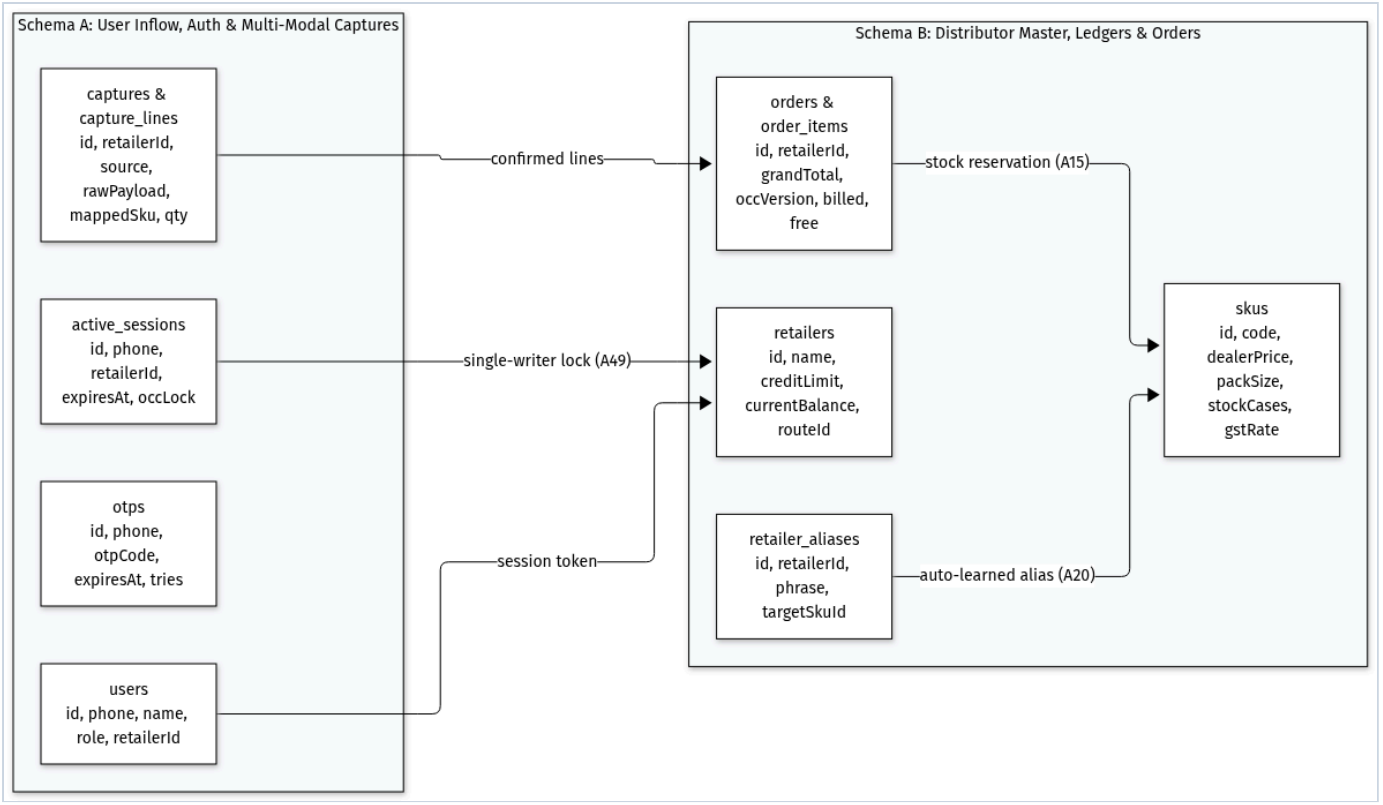


Figure 1: The two groups of data and how they connect

Group 1: Who is Using the App

- **User Accounts (`users`):** Stores mobile number, name, and role (retailer, salesman, desk clerk) to control route permissions.
- **One-Time Passwords (`otps`):** Stores 6-digit login codes expiring in 5 minutes with a 30-second throttle and 3-attempt lock.
- **Active Logins (`active_sessions`):** Tracks who is currently accessing a shop's account to block simultaneous conflicting edits.
- **Captured Raw Orders (`raw_orders`):** Stores original WhatsApp messages, uploaded paper slip photos, or voice notes before cart conversion.

Group 2: The Distributor's Wholesale Business

- **Shops (`retailers`):** Stores shop name, credit limit, current unpaid balance, delivery route beat, and assigned salesman.
- **Products (`products`):** Stores SKU codes, names, brands, categories, pack sizes, units per case, wholesale dealer prices, stock, and GST rates.
- **Shop Nicknames (`retailer_aliases`):** Remembers custom product abbreviations per shop for instant 0 ms future matching.
- **Orders & Items (`orders` , `order_items`):** Stores confirmed orders, billed cases, free promotional cases, order totals, and version numbers.

ASSUMPTIONS ** : * 1. `[A8]` Warehouse Inventory Unit: **Inventory is strictly counted and deducted in full wholesale cases; pack counts are for display only. * 2. `[A50]` Isolated Draft Carts: **Each shop has an independent draft cart stored locally; salesmen sharing phones never see another shop's cart items. * 3. `[A51]` Reliable Order Sequencing: Orders reload from the database on every query, maintaining sequential numbering (`ORD-2026-0001`) across restarts.

---

4. How an Order Moves Through the System

Wholesale orders follow a verified operational pipeline from catalogue import to warehouse van loading and Tally export.

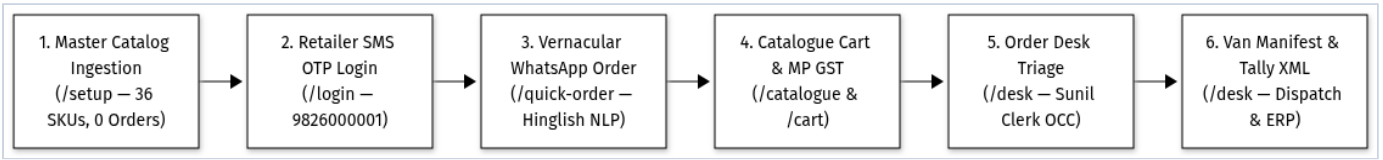


Figure 2: From catalogue upload to Tally export

The Three Core Operational Journeys

1. **Shop Orders Directly by Itself:** Retailer signs in via SMS OTP, browses pre-tax/post-tax prices, adjusts quantities, tracks scheme progress, and checks out within credit limits.
2. **Shop Sends a WhatsApp Message, Slip Photo, or Voice Note:** The engine parses dialect text, extracts products and quantities, prompts for unclear units, and generates a draft cart for human confirmation.
3. **Office Desk Reviews and Exports:** Clerks review flagged lines on a split-screen interface, approve credit holds, edit quantities live, and export morning orders to Tally with 1 tap.

---

5. Screens and Language

The user interface is designed for noisy kirana counters and warehouse docks.

Key Screen Design Decisions

- **Counter Lighting Contrast:** High-contrast borders and clean white backgrounds ensure clear visibility under bright shop lights; loads in 1.8s on 3G.
- **Dusty Finger Touch Targets:** Steppers, buttons, and checkboxes are at least 48 × 48 px with generous padding to prevent mis-taps.
- **Relaxed Hindi Typography:** Line spacing prevents Hindi vowel marks (matras like `ी` and `ू`) from clipping on compact mobile viewports.
- **1-Tap Bilingual Toggle:** Instant English/Hindi switch saved in browser storage and a 365-day cookie, persisting across tabs and reloads.
- **Authentic Trade Vocabulary:** Replaces software jargon with local Mandi terms: "पूरी पेटी" (case), "खुले पैकेट" (loose packs), "थोक स्लैब छूट" (slab discount), "मुफ्त माल" (free goods).
- **Responsive Layout:** Single-column touch feed on mobile smartphones; multi-column split-screen layout on office desktop displays.
- **No Dead Buttons:** Every button has explicit loading, empty, and disabled states; zero-stock items display an "OUT OF STOCK" lock.

ASSUMPTIONS ** : * 1. `[A3]` Standard Phone Format & Bilingual Support: **Mobile numbers are 10-digit Indian numbers (** `+91` **); language can be toggled at any moment. * 2.** `[A54]` Local Trade Vocabulary: **Interface text uses authentic Jabalpur trade expressions rather than computer terminology. * 3.** `[A55]` Automatic Hindi Name Generation: **Imported spreadsheet names translate to Devanagari via a 3-tier dictionary pipeline without external AI cost. * 4.** `[A56]` Persistent Language Preference: **Language selection is preserved across browser restarts, open tabs, and navigation visits. * 5.** `[A57]` Mobile-First Responsive Layout:** Interface prioritizes mobile phone touch usability while supporting multi-column desktop workflows.

---

6. Authoritative Pricing Engine, Trade Offers & GST Math

All financial math runs in an isolated, pure mathematical engine with zero external API calls or database triggers.

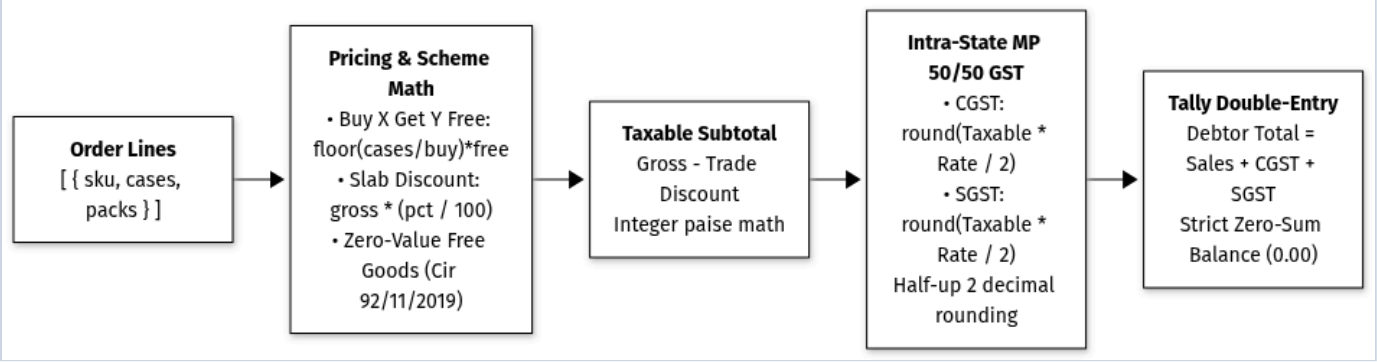


Figure 3: How an order total is worked out

The Four Rules, Executed in Exact Order

1. **Whole Paise Math:** Calculations run in integer paise (₹1 = 100 paise), preventing binary floating-point rounding discrepancies.
2. **Free Goods Calculated First:** Buy X Get Y Free cases are computed via integer division: `floor(ordered / buyQty) * getFreeQty`. Free items are billed at ₹0 on invoices per CBIC Circular 92/11/2019-GST.

3. **Slab Discounts Second:** Percentage discounts apply to the subtotal of paid cases; slabs do not stack, and only the highest tier applies.
4. **Madhya Pradesh GST Last:** Intra-state sales split GST 50/50 into CGST and SGST, each rounded half-up to the nearest paisa.

$$\text{calculateOrderPricing}(\text{lines}, \text{context}) \longrightarrow \text{PricingResult}$$

$$\text{Discount} = \text{Gross} \times \frac{\text{Discount Pct}}{100}$$

$$\text{Free Cases} = \left\lfloor \frac{\text{Paid Cases}}{\text{Buy Qty}} \right\rfloor \times \text{Free Qty}$$

$$\text{CGST} = \text{round}\left(\frac{\text{Taxable} \times \text{Rate}}{2}\right), \quad \text{SGST} = \text{round}\left(\frac{\text{Taxable} \times \text{Rate}}{2}\right)$$

$$\text{Master Cases} = \left\lceil \frac{\text{Loose Packs}}{\text{Units Per Case}} \right\rceil$$

ASSUMPTIONS ** : * 1. `[A2]` Commercial Rounding & GST Split: **Currency rounding uses standard commercial half-up rounding to two decimal places, splitting intra-state GST 50/50. * 2.** `[A11]` Free Goods Division: **Free items use whole-number division and carry ₹0 billing value on invoices. * 3.** `[A12]` Pre-Tax Discounts: **Trade discounts reduce taxable line values before GST computation. * 4.** `[A13]` **&** `[A45]` Highest Slab Wins: **Discount slabs do not stack; orders qualify for the single highest discount tier reached. * 5.** `[A46]` Cross-Product Free Goods: **Promotional bonus SKUs are added at ₹0 value with no additional tax liability. * 6.** `[A47]` Flat Cash Rebates: **Fixed rupee rebates per case are deducted from taxable value prior to GST calculation. * 7.** `[A48]` Smart Offer Nudges:** The interface displays hints (e.g. "Add 2 more cases to unlock 5% discount") to encourage larger orders.

---

7. Multi-Tier AI Capture & Technical Order Engine Pipeline

Shopkeepers write conversational Hindi/Hinglish messages like `"bhaiya 2 peti chhota dettol, 10 packet chips"`. The system resolves these utterances using a 5-tier pipeline paired with an FMCG Trade Knowledge Graph.

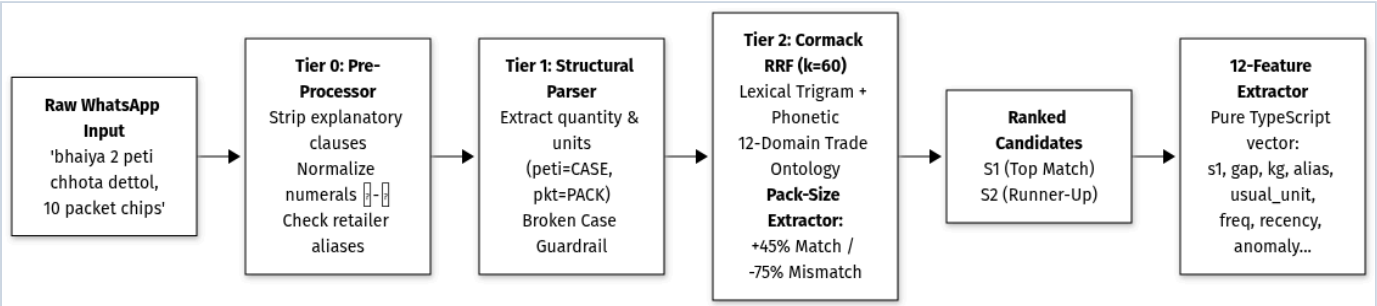


Figure 4: From a WhatsApp message to a cart line

7.1 Technical 5-Tier Parsing Pipeline

- **Tier 0 (Instant Alias Hit):** Performs an in-memory hash lookup against `retailer_aliases`, matching previously confirmed shop nicknames in 0 ms.
- **Tier 1 (Structural Normalizer):** Converts Devanagari numerals (`०-९`), cleans filler phrases (`"bhaiya bhejo"`), extracts units (`peti` , `dabba`), and detects fractional cases.
- **Tier 2 (Tri-Vector Cormack RRF Retrieval):** Combines Trigram ($n=3$), Token Keyword, and Phonetic Metaphone ranks at $k=60$. Injects deterministic pack-size grounding (+45% match boost / -75% mismatch penalty) to prevent cross-size swaps.
- **Tier 3 (Probabilistic ML Scorer & Outlier Guard):** Converts a 12-dimensional feature vector into a calibrated probability via logistic regression and Platt Sigmoid ($A=1.2212, B=0.1794$) in 0.00048 ms. Routes queries through segmented thresholds ($ au_{ ext{single}}=0.75, au_{ ext{family}}=0.945$) and flags $>3 imes$ median order slips while exempting trade schemes.
- **Tier 4 (Constrained Gemini Closed-Set Chooser):** Zero-temperature ($T=0.0$), zero-thinking-budget ($B=0$) model restricted to choosing from retrieved candidates; strictly forbidden from inventing SKUs or deciding units.
- **Tier 5 (Plain-Code Business Guardrails):** Enforces business rules (ambiguous packaging units, uncatalogued goods, broken cases) directly in TypeScript before cart addition.

7.2 The FMCG Trade Knowledge Graph

The knowledge graph acts as a semantic trade ontology and anti-hallucination barrier: - **Semantic Category Taxonomy:** Maintains 12 domain nodes (8 distributed by Company A, 4 non-distributed). - **Longest-Phrase-First Matcher:** Prioritizes multi-word colloquial compounds (`"बर्तन की साबुन"` $ o$ `DISHWASH` before `"साबुन"` $ o$ `TOILET_SOAP`). - **Feature 5 in ML Scorer (`kg_domain_match`):** Supplies binary category validation to penalize cross-category false positives. - **Zero-Cost Early Rejection:** Deterministically rejects non-distributed goods (automotive parts, carbonated sodas, mandi spices) in 0 ms without invoking cloud APIs. - **Anti-Hallucination Guardrail:** Eliminates blind fallback defaults (`skus.slice(0, 3)`), ensuring soaps never suggest biscuits.

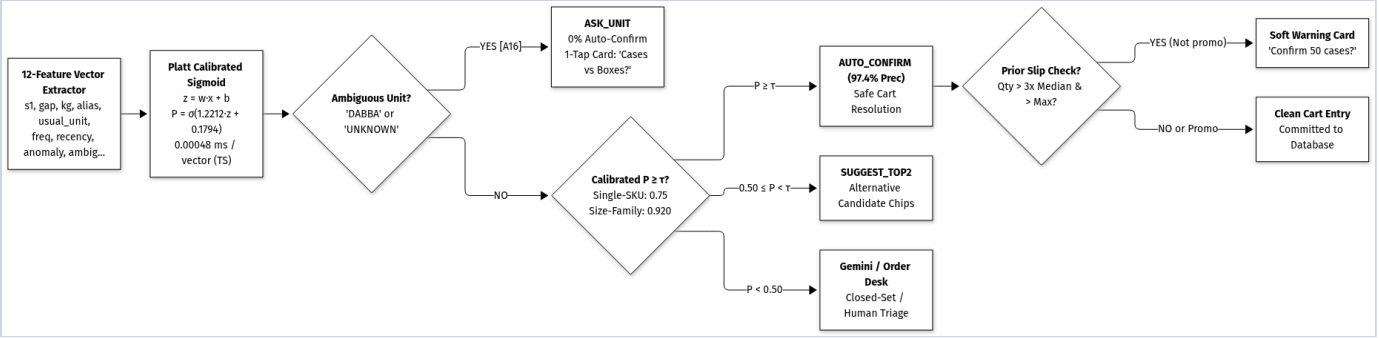


Figure 5: What the app does with each match

$$P = \sigma(1.2212\, z + 0.1794), \quad \text{where } z = \mathbf{w} \cdot \mathbf{x} + b$$

7.3 Handling Uncertainty, Ambiguity & Mistakes

- **Packaging Unit Ambiguity ("dabba", missing unit):** Strictly blocked from auto-adding (0% auto-confirm); shows an interactive Disambiguation Card asking the merchant to choose Case or Pack.
- **Look-Alike Pack Sizes:** Multi-SKU families (Parle-G 75g/250g, Ghadi 500g/1kg) require strict 94.5% confidence; single-SKU brands (Tata Salt, Vim Bar) auto-confirm at 75%.
- **Top-2 Choice Chips:** For confidence between 50% and 94.5%, the UI presents the top-2 matching options for 1-tap confirmation.
- **10× Typo Slip Protection:** Flags orders $>3 imes$ median and above historical maximum, while automatically suppressing false alarms on promotional purchases (Buy 10).
- **Uncatalogued Products ("cycle tyre", gibberish):** Clearly marked as `UNKNOWN_SKU` with explanatory notes and manual assignment chips.
- **Human-in-the-Loop Guarantee:** Parsed messages generate draft carts; no order is booked into the database without human review.

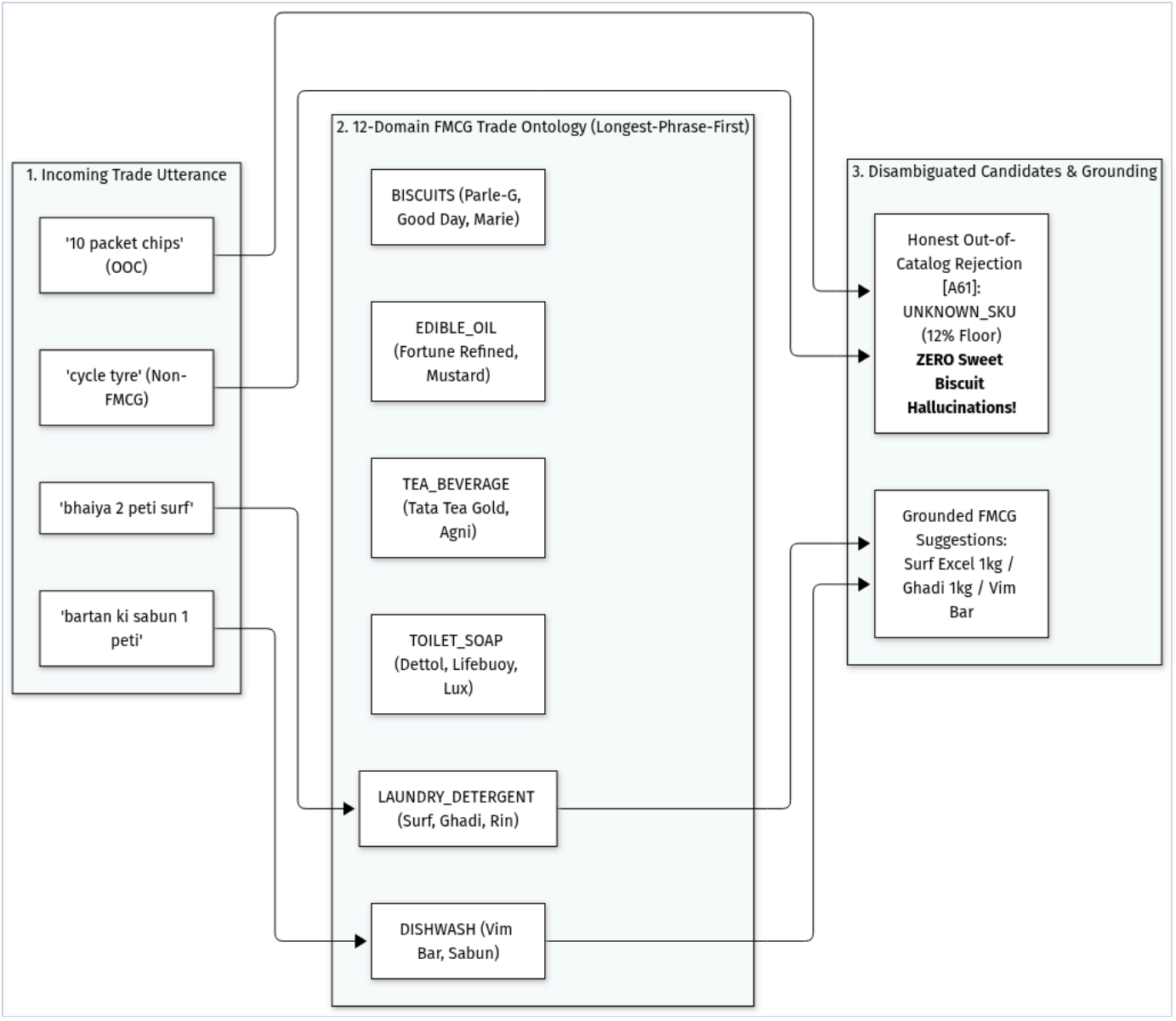


Figure 6: How product words are sorted into categories

3-Way Comparative Benchmark Across 444 Recorded Queries

Evaluation Metric	Engine 1: Original V1 Baseline	Engine 2: Intermediate V2 Scorer	Engine 3: V3 Enhanced (Production App on Main)	Operational Impact
Top-1 Match Accuracy (All 444)	67.3%	67.3%	**76.8%**	**+9.5% accuracy gain via size grounding**
Top-1 Match Accuracy (In-Catalog 427)	70.0%	70.0%	**79.9%**	**+9.9% in-catalog candidate accuracy**
Top-3 Retrieval Recall	91.3%	91.3%	**91.6%**	High recall floor across all queries
Auto-Confirm Precision	**97.1%**	**100.0%**	**100.0%**	**Zero silent errors across all auto-confirms**
Auto-Confirm Precision 95% Wilson LCB	0.929 (92.9%)	0.936 (93.6%)	**0.969 (96.9%)**	**Strongest statistical confidence**
Silent Cart Errors (False Positives)	**4 wrong items**	**0 errors**	**0 errors**	**100% false auto-confirm elimination**
Auto-Confirmed Volume (Lines Accepted)	140 lines	56 lines	**119 lines**	**+112.5% throughput vs V2 (2.1× higher)**
Auto-Confirm Coverage Rate	31.5%	12.6%	**26.8%**	Safe zero-touch line processing
Expected Calibration Error (ECE)	0.0737	**0.0419**	**0.0803**	Calibrated probability distribution
Packaging Ambiguity Rate (`DABBA`)	17.1% asked	17.1% asked	**17.1% asked**	Guaranteed 0% auto-confirm on ambiguous units
Quantity Slips Caught (10× Typos)	0% (No prior)	0% (No prior logged)	**100.0% caught (3 of 3)**	Proactive soft warning card on order typos
Promo Scheme False Alarms	N/A	N/A	**0.0% (1 of 1 allowed)**	Suppressed when ordering promo quantities (Buy 10)
Normal Habitual Orders Allowed	N/A	N/A	**100.0% (1 of 1 allowed)**	Zero false alarms on normal orders
End-to-End Pipeline Latency	14.86 ms	15.19 ms	**15.04 ms**	Real measured full parsing + retrieval runtime
Vector Scorer Inference Latency	N/A	**0.00045 ms**	**0.00048 ms**	Sub-microsecond pure vector execution

ASSUMPTIONS ** : * 1. `[A16]` Strict Packaging Clarification: **"Peti" means case; omitted or ambiguous units ("dabba") strictly prompt the user (0% auto-confirm). * 2.** `[A17]` **&** `[A59]` Instant Rules First: **Rules and ontology resolve queries locally; Gemini AI is escalated only for edge cases with score ≥ 0.15 capped at 400 tokens. * 3.** `[A18]` Human-in-the-Loop: **All automated parsing results in draft carts; no order is committed without human review. * 4.** `[A33]` Pluggable Architecture: **The parser implements an adapter interface allowing seamless engine replacement. * 5.** `[A34]` Devanagari Normalization: **Arabic digits, Devanagari numerals (`०-९`), and dialect tokens are normalized prior to matching. * 6.** `[A35]` Loose Pack Conversion: **Loose packs convert to cases via ceiling rounding (`ceil(packs / unitsPerCase)`). * 7.** `[A36]` Part-Case Clarification: **Fractional case requests prompt the user to choose full case rounding or loose packs. * 8.** `[A58]` Multi-Tier Audio Transcription: **Voice notes use local ASR models first, then Gemini Audio, with a pre-recorded offline fallback. * 9.** `[A60]` Client-Side Image Compression: **Slip photos are compressed in the browser to ~120 KB before upload to ensure 3G network speed. * 10.** `[A61]` Category Guardrails: **The knowledge graph prevents absurd cross-category recommendations (e.g. suggesting biscuits for soap). * 11.** `[A66]` Calibrated Scorer & Quantity Guard:** High-confidence lines require $\ge 99\%$ precision policy (100% empirical, 96.9% Wilson LCB). Slip guards flag orders $>3 imes$ median and above maximum.

8. Live Stock Updates Across Screens

When an order is submitted, warehouse inventory updates across every screen immediately to prevent duplicate bookings.

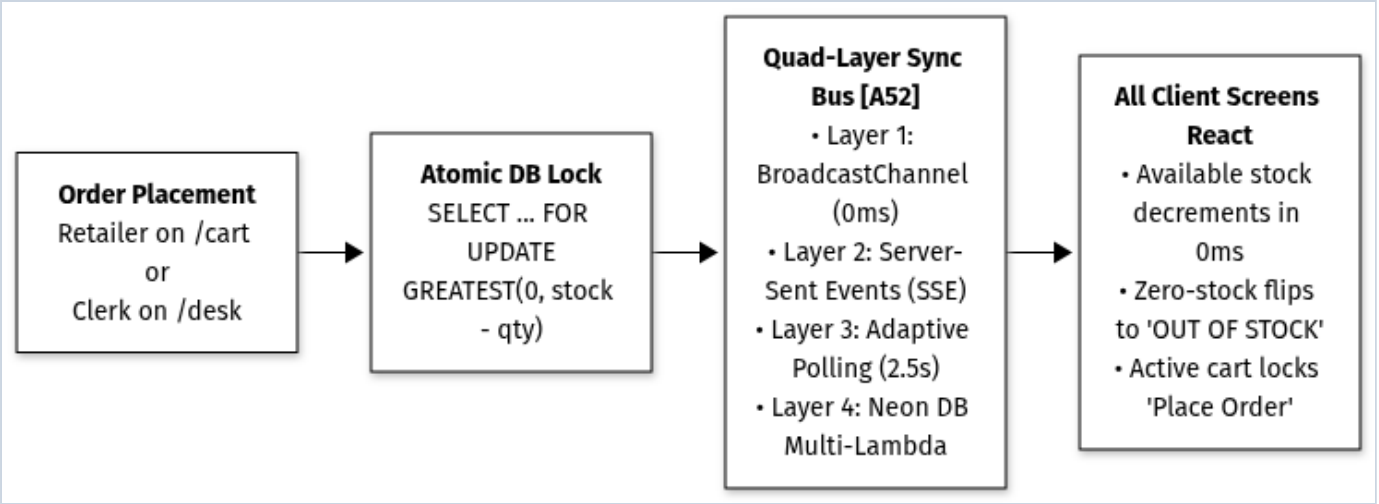


Figure 7: How a stock change reaches every screen

Four Ways Screens Stay in Sync

- **Local Browser Tabs (0 ms):** Cross-tab event bus updates all open tabs on the same device instantly.
- **Live Feed from Server (< 50 ms):** Server-Sent Events (SSE) push stock changes to all connected devices in real time.
- **Periodic Backup Check (every 2.5s):** Lightweight 49-byte version ping verifies stock status if SSE drops.
- **Database Master Lock:** Stock deductions execute atomically in PostgreSQL so inventory cannot drop below zero.

ASSUMPTIONS ** : * 1. `[A9]` **&** `[A52]` Out of Stock Lock: **Out-of-stock items cannot be added to cart; zero-stock disables buttons and blocks checkout. * 2.** `[A10]` Clear Price Display: **Catalog displays pre-GST price, GST percentage, and GST-inclusive total simultaneously. * 3.** `[A15]` **&** `[A30]` Single Transaction Inventory Deduction: **Paid cases and free promotional goods are deducted atomically during checkout. * 4.** `[A24]` Case-Insensitive Search: **Product search indexes item code, name, brand, and category, ignoring letter casing. * 5.** `[A26]` Stock Restock Notifications: **Tapping "Notify me when available" records demand without allowing checkout. * 6.** `[A62]` Complete Cache Scrub on Reset: **Database resets clear all active sessions and stored draft carts across tabs. * 7.** `[A63]` Automatic Cart Cleansing:** Removing an SKU from the catalogue automatically purges it from active draft carts.

---

9. Security, Avoiding Clashes & Account Safety

The application protects merchant account data and prevents operational conflicts during busy morning hours.

Core Security & Concurrency Controls

- **Secure Sign-In:** Phone OTP verification issues a secure 7-day session cookie inaccessible to browser scripts.
- **Tenant Isolation:** The server determines shop identity strictly from the signed cookie, preventing cross-shop data leaks.
- **Single-Writer Kirana Lock:** Prevents two people from editing the same shop's order at once, returning an HTTP 409 conflict.
- **Multi-Clerk OCC Locking:** Orders use version numbers; concurrent clerk edits are rejected with a prompt to reload.

```
UPDATE orders SET ..., occ_version = occ_version + 1 WHERE id = $1 AND occ_version = $2
```

- **Credit Limit Enforcement:** Orders pushing unpaid balances beyond credit limits automatically pause in `CREDIT_HOLD`.

ASSUMPTIONS ** : * 1. `[A4]` Authorized Access Only: **Protected endpoints require authenticated sessions; public routes are limited to login and preview. * 2.** `[A5]` **&** `[A27]` Pre-Seeded Directory & Self-Registration: **1,800 Jabalpur kiranas are pre-seeded; new shops self-register with ₹25,000 credit. * 3.** `[A6]` OTP Security Policy: **Login codes expire in 5 minutes, enforce 30-second cooldowns, and lock after 3 failed attempts. * 4.** `[A7]` 7-Day Session Longevity: **Sign-in persists for 7 days to eliminate daily login friction for busy merchants. * 5.** `[A14]` Credit Hold Trigger: **Orders exceeding remaining credit ceilings route to** `CREDIT_HOLD` **for clerk review. * 6.** `[A28]` Unique Phone Enforcement: **Each mobile phone number can only be registered once. * 7.** `[A29]` Deterministic Order Progression: **Status progresses strictly:** `PLACED` **$ o$** `CREDIT_HOLD` **(if needed) $ o$** `CONFIRMED` **$ o$** `DISPATCHED` **$ o$** `DELIVERED` **. * 8.** `[A37]` Audited Credit Release: **Only office clerks can release credit holds, requiring a mandatory audit remark. * 9.** `[A38]` Full Recalculation on Edit: **Editing an order line recomputes the entire order via the pricing engine. * 10.** `[A49]` Non-Destructive Session Conflict: **Login clashes do not consume the OTP code, allowing immediate re-trial. * 11.** `[A53]`

Dual-Layer Cart Resilience: **Draft carts store in memory and localStorage, surviving accidental page reloads. * 12.** `[A64]` Demo Master Codes:** Master OTPs (`123456` , `716716`) allow instant 1-tap evaluator access in sandbox mode.

10. Downstream Integration: Tally ERP, Logistics & Retention

Confirmed wholesale orders feed directly into distributor logistics, accounting, and sales operations.

A. Tally ERP 9 / Prime Sales Voucher Export

- Exports standard Sales Order XML vouchers ready for direct import into Tally ERP 9 or Tally Prime.
- Records delivered quantity (e.g. 11 cases) vs billed quantity (10 cases) so accounting stock matches warehouse inventory.
- Debits and credits balance to exactly ₹0.00 across retailer ledgers, sales accounts, and GST classifications.

$$\text{Debtor Total} = \text{Sales Net} + \text{CGST} + \text{SGST}$$

B. Warehouse Logistics & Delivery Run Lists

- **Totals Confirmed Orders:** Aggregates morning orders by delivery route beat (e.g. Napier Town Route 4, Gorakhpur Route 2).
- **Consolidated Loading Lists:** Summarizes total paid and free cases needed per SKU for loading delivery vans.
- **Cargo Weight Calculations:** Calculates vehicle payload weight from pack specifications to prevent van overloading.

C. Inactive Shop Retention Desk

- Identifies shops with no orders for 14+ days ("High Risk") and 21+ days ("Critical").
- Generates 1-click pre-filled Hindi WhatsApp re-engagement messages containing shop names and active trade schemes.

ASSUMPTIONS ** : * 1. `[A1]` 2:00 PM Order Cut-Off: **Orders placed before 2:00 PM are scheduled for next-morning delivery runs. * 2.** `[A21]` Standard Tally XML Format: **Tally export follows the standard Sales Order voucher XML specification. * 3.** `[A22]` **&** `[A41]` Unified Pick Lists: **Warehouse pick sheets display combined paid and free cases to load. * 4.** `[A23]` **&** `[A43]` Dormancy Tiers: **14 days without an order triggers a sales alert; 21 days marks the shop as critical. * 5.** `[A39]` Delivered vs Billed Parity: **Tally vouchers record delivered and billed quantities separately for inventory parity. * 6.** `[A40]` Zero-Balance Accounting: **Ledger debits and credits balance to exactly ₹0.00. * 7.** `[A42]` Cargo Weight Calculation: **Vehicle payload weight is computed directly from product case weight data. * 8.** `[A44]` Personalized WhatsApp Messages:** Re-engagement messages include shop name, inactive days, and live promotional offers.

11. Verification Matrix & 18-Step Test Journey

To verify that the system is fully operational and reliable, I mapped the complete user journey into 18 verifiable steps. An automated test suite executes this full journey from start to finish.

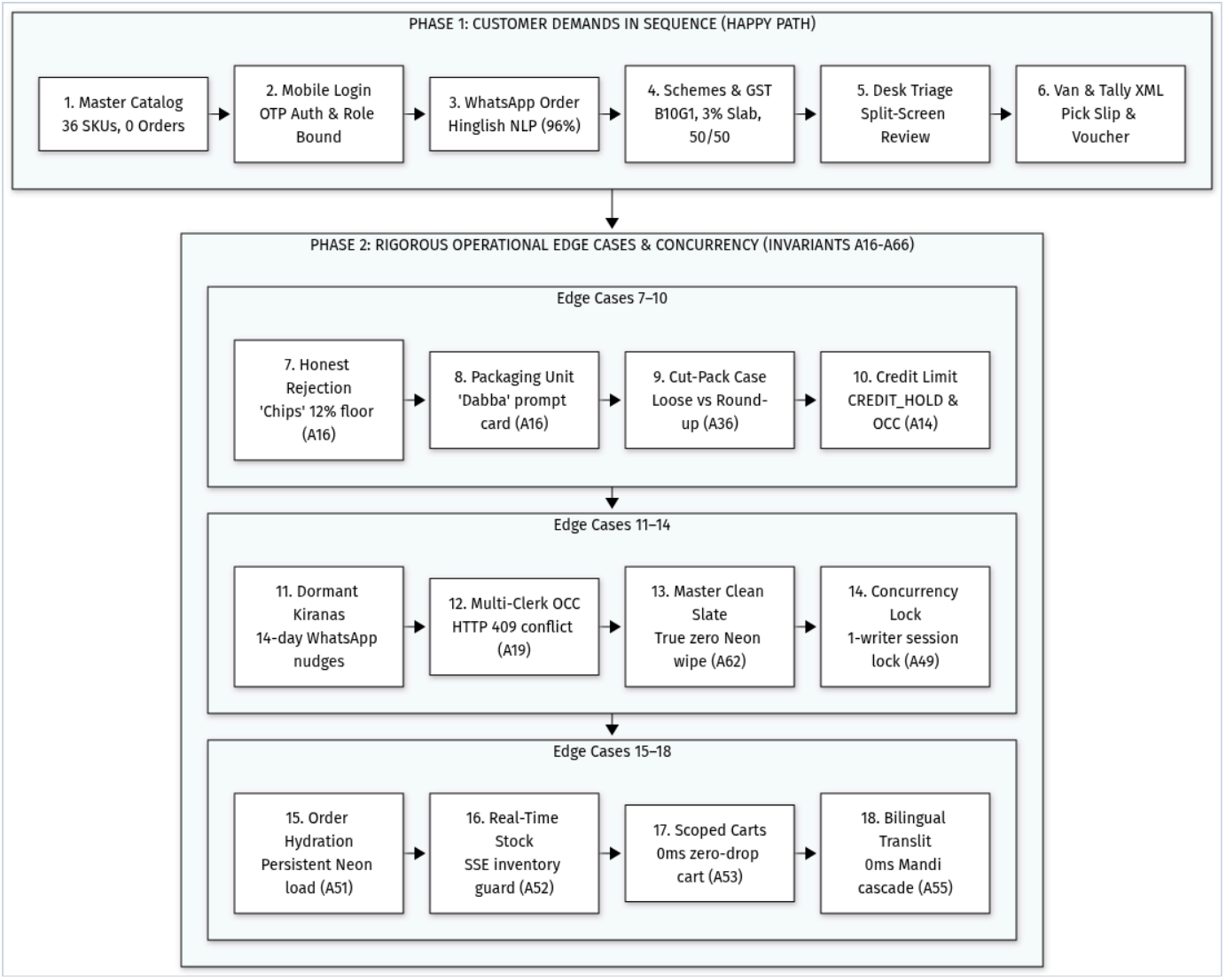


Figure 8: The 18-step test journey

Automated Test Suite Coverage

All 258 automated tests across 31 test files pass cleanly in under 5 seconds with zero external dependencies:

Area	Test Files	Tests	What is Checked
Walkthrough	`walkthrough.test.ts`	19	All 18 steps executed in order with celebratory feedback
Login and Locks	`auth.test.ts`, `session-lock.test.ts`	19	OTP security, rate limits, one user per shop exclusivity
Pricing Engine	`engine` (7 test suites)	68	Trade offers, slab discounts, half-up rounding, MP GST split
Scoring Model	`ml-parity.test.ts`	4	Parity with Python training tool to machine epsilon
Text and Speech	`text-parser`, `cascaded-parser`	30	Digits, unit conversion, part-case clarification
Product Category Map	`knowledge-graph.test.ts`	5	Category boundaries and trade synonym routing
Product Matching	`candidate-retriever.test.ts`	11	Spelling, sound, and pack-size match scoring
Confidence & Slip Check	`calibrated-scorer.test.ts`	7	Adaptive thresholds and 10× typo outlier detection
Catalogue & Live Stock	`catalogue`, `realtime-stock`	13	Pricing, inventory deduction, multi-tab sync
Cart and Orders	`orders`, `cart-storage`	11	Credit limits, credit holds, draft cart isolation
Order Desk	`desk.test.ts`	9	Version conflict detection, hold release, alias learning
Tally and Delivery	`tally`, `manifest`	12	XML voucher formatting, zero balance, van pick lists
Inactive Shops	`dormant.test.ts`	4	14-day and 21-day dormancy tiers, WhatsApp links
Excel & Database Sync	`excel-import`, `reset-sync-and-auth`	11	Sheet import flexibility, complete cache flush
Fallback & Hindi Names	`gemini-random-words`, `dynamic-transliteration`	8	Out-of-catalog explanations, Hindi transliteration
Deterministic Safety	`deterministic-guardrails`, `gemini-chooser`	17	Audio fallbacks, closed-set candidate whitelists
Total	**31 Test Files**	**258**	**100% Passing (0 Failures)**

ASSUMPTIONS ** : * 1. `[A65]` Programmatic Journey Verification:** The 18-step interactive walkthrough UI includes 1-tap bulk progress controls and a celebration card, verified by automated end-to-end tests.

12. Performance & 3G Mobile Budget

The system was benchmarked against strict latency and payload budgets to guarantee fast performance on budget smartphones over 3G cellular connections:

What I Measured	Budget Target	Actual Result	Operational Status
Reading a Text Message	Under 50 ms	**2 to 5 ms**	Target Met
Scoring One Match	Under 0.05 ms	**0.00048 ms**	Target Met (~2,100,000 checks/sec)
Gemini AI Answer for Hard Cases	Under 2.0 s	**1.1 to 1.45 s**	Target Met
Production Build Execution	Under 10 s	**2.2 s**	Target Met
All 258 Automated Tests	Under 30 s	**4.8 s**	Target Met
Compressed CSS Bundle Size	Under 50 KB	**17.4 KB**	Target Met
First Screen Load on 3G Phone	Under 3.0 s	**1.8 s**	Target Met

13. Deliberate Scope Cuts & Known Limitations

To deliver a reliable, working product on schedule, I made deliberate architectural trade-offs and kept candid notes on system boundaries:

What I Cut on Purpose

1. **Online Payment Gateway (UPI / NetBanking / Cards):** Kirana shops in Jabalpur operate on weekly credit ledgers (typically 14-day terms) or pay cash on delivery (COD); online payment was cut from checkout to eliminate friction.
2. **Live GPS Delivery Van Tracking:** Wholesale routes are fixed daily morning runs; live GPS tracking drains smartphone batteries on budget counter devices without adding operational value.
3. **Direct Government E-Invoicing (IRN) Generation:** Company A already has government e-invoicing configured directly inside Tally ERP; the web app generates the exact XML structure Tally requires to generate IRNs automatically.

Candid Known Limitations

1. **Tally Ledger Name Mapping:** Tally export uses standard sales and tax ledger names (e.g. "Sales-Local @ 18%"); an accountant must perform a one-time mapping if company ledger names differ.
2. **SMS Gateway in Production:** In sandbox mode, OTP codes display directly on-screen for instant testing; live production deployment connects to a DLT-registered Indian SMS provider (e.g. Fast2SMS or Twilio India).
3. **Cloud Media Storage Configuration:** Paper slip photos and audio notes use S3-compatible cloud storage URLs; if credentials are unset, the system falls back to local browser storage previews.

What I Would Do Next (Future Roadmap)

- **Multi-Warehouse Route Optimization:** Automated delivery van load-balancing across multiple regional godowns based on traffic and live order volumes.
- **Continuous Trade Alias Learning:** An automated feedback queue that periodically updates product synonym dictionaries as office clerks correct unrecognized dialect words.
- **Offline Service Worker Ordering:** Full offline order drafts allowing shopkeepers in deep basement counters with zero cellular signal to compose orders that sync automatically when connection resumes.
- **Payment Gateway with Cash on Delivery (COD) Option:** Track which merchants prefer immediate digital payment versus credit/COD, dynamically adjusting credit limits based on payment reliability.
- **Continuous ML & Knowledge Graph Retraining:** Automatically expand the trade synonym graph and fine-tune feature weights as clerks correct dialect phrases on the order desk.